

Como criar minha primeira classe em Python

Aprenda neste conteúdo como criar sua primeira classe utilizando a linguagem de desenvolvimento Python.

Criar uma classe em Python é bastante simples.



```
01 class Pessoa:
02     def __init__(self, nome, idade):
03         self.nome = nome
04         self.idade = idade
05
06     def setNome(self, nome):
07         self.nome = nome
08
09     def setIdade(self, idade):
10         self.idade = idade
11
12     def getNome(self):
13         return self.nome
14
15     def getIdade(self):
16         return self.idade
```

Para definir o nome da classe usamos a palavra reservada **class**.

```
01 class Pessoa:
02
03
04
05
06
07
08
09
10
11
12
13
14
15
```

Após o nome da classe devemos adicionar dois pontos :

Para criar um método
usamos a palavra reservada
def.

O construtor é um método
reservado chamado
__init__.

O parâmetro **self** é
obrigatório e os demais
são definidos por nós.

```
01 class Pessoa:
02     def __init__(self, nome, idade):
03
04
05
06
07
08
09
10
11
12
13
14
15
```

Aqui está o corpo do
método, sempre indentado
como manda a sintaxe da
linguagem.

```
01 class Pessoa:
02     def __init__(self, nome, idade):
03         self.nome = nome
04         self.idade = idade
05
06
07
08
09
10
11
12
13
14
15
```

**A partir daí podemos definir
quantos métodos precisarmos.**



```
01 class Pessoa:
02     def __init__(self, nome, idade):
03         self.nome = nome
04         self.idade = idade
05
06     def setNome(self, nome):
07         self.nome = nome
08
09     def setIdade(self, idade):
10         self.idade = idade
11
12     def getNome(self):
13         return self.nome
14
15     def getIdade(self):
16         return self.idade
```

#PraCegoVer - Transcrição dos Slides

Criar uma classe em Python é bastante simples

Para definir o nome da classe usamos a palavra reservada `class` - `class Pessoa`: - Após o nome da classe devemos adicionar dois pontos

Para criar um método usamos a palavra reservada `def`. O construtor é um método reservado chamado `__init__`. O parâmetro `self` é obrigatório e os demais são definidos por nós. - `def __init__(self, nome, idade)`:

```
    Aqui está o corpo do método, sempre indentado como manda a sintaxe da linguagem - self.nome =  
    nome  
  
    self.idade = idade
```

A partir daí podemos definir quantos métodos precisarmos.

Primeira classe em Python

No **Python** a **criação de classes** é bem simples e nos permite definir quais atributos e métodos uma classe irá possuir. Na **Figura 1** podemos visualizar a representação de uma classe com o nome `Pessoa` e quais atributos esta possui:

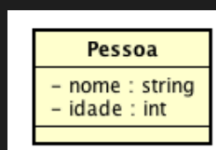


Figura 1. Diagrama de classe Pessoa

Abaixo podemos ver como a classe citada acima seria implementada utilizando a linguagem Python:

```
1 class Pessoa:
2     def __init__(self, nome, idade):
3         self.nome = nome
4         self.idade = idade
5
6     def setNome(self, nome):
7         self.nome = nome
8
9     def setIdade(self, idade):
10        self.idade = idade
11
12    def getNome(self):
13        return self.nome
14
15    def getIdade(self):
16        return self.idade
```

Linha 1: A criação de uma classe começa pelo uso da palavra reservada `class`, seguida do nome da classe e dois pontos;

Linha 2: Aqui temos a definição do construtor da classe, que é um método especial chamado `__init__`. Como todo método em Python, sua declaração começa com `def` e entre parênteses estão os parâmetros, incluindo o parâmetro obrigatório `self`, que está presente em todos os métodos;

Linhas 3 e 4: O corpo do método deve estar identado, como manda a sintaxe da linguagem. Aqui estamos apenas atribuindo os valores recebidos por parâmetro aos atributos da classe;

Linhas 6 a 16: Criamos os métodos get e set de todos os atributos da classe Pessoa que serão responsáveis, respectivamente, por retornar ou modificar os atributos desta classe.

Herança em Python

Também podemos implementar herança entre classes no Python.



```
01 from pessoa import Pessoa
```

```
02
```

```
03
```

```
04
```

```
05
```

```
06
```

```
07
```

```
08
```

```
09
```

```
10
```

```
11
```

```
12
```

```
13
```

```
14
```

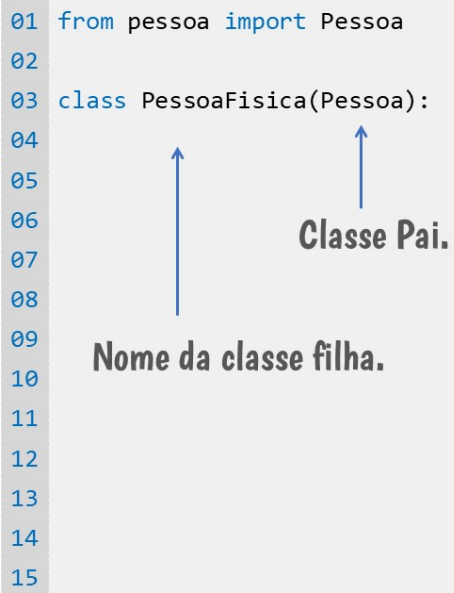
```
15
```

Nome da classe Pai.

Nome do arquivo em que a classe pai foi criada (sem a extensão .py).

Primeiro precisamos importar a classe pai.

```
01 from pessoa import Pessoa
02
03 class PessoaFisica(Pessoa):
04
05
06
07
08
09
10
11
12
13
14
15
```

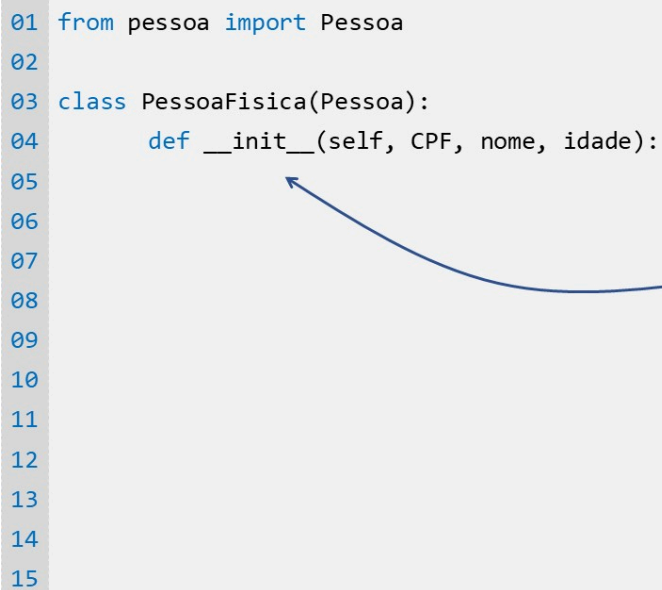


Nome da classe filha.

Classe Pai.

Em seguida podemos definir a classe filha herdando da classe pai (entre parênteses).

```
01 from pessoa import Pessoa
02
03 class PessoaFisica(Pessoa):
04     def __init__(self, CPF, nome, idade):
05
06
07
08
09
10
11
12
13
14
15
```



Podemos criar um novo construtor.

```
01 from pessoa import Pessoa
02
03 class PessoaFisica(Pessoa):
04     def __init__(self, CPF, nome, idade):
05         super().__init__(nome, idade)
06
07
08
09
10
11
12
13
14
15
```

E invocar o construtor da classe pai, aproveitando seu funcionamento padrão.

```
01 from pessoa import Pessoa
02
03 class PessoaFisica(Pessoa):
04     def __init__(self, cpf, nome, idade):
05         super().__init__(nome, idade)
06         self.CPF = cpf
07
08
09
10
11
12
13
14
15
```

A classe filha pode ter seus próprios atributos...

```
01 from pessoa import Pessoa
02
03 class PessoaFisica(Pessoa):
04     def __init__(self, cpf, nome, idade):
05         super().__init__(nome, idade)
06         self.CPF = cpf
07
08     def setCPF(self, cpf):
09         self.CPF = cpf
10
11     def getCPF(self):
12         return self.CPF
13
14
15
```

...e também seus próprios métodos.



#PraCegoVer - Transcrição dos Slides

Também podemos implementar herança entre classes no Python: PessoaFisica e PessoaJuridica herdam de Pessoa

Primeiro precisamos importar a classe pai - `from pessoa import Pessoa` - pessoa é o nome do arquivo em que a classe pai foi criada (sem a extensão .py)
Pessoa é o nome da classe Pai.

Em seguida podemos definir a classe filha herdando da classe pai (entre parênteses) - `class PessoaFisica(Pessoa):` - onde PessoaFisica é o nome da classe filha e Pessoa é a classe pai

Podemos criar um novo construtor: `def _init_(self, CPF, nome, idade):`

E invocar o construtor da classe pai, aproveitando seu funcionamento padrão - `super()._init_(nome, idade)`

A classe filha pode ter seus próprios atributos... - `self.CPF = cpf`

... e também seus próprios métodos:

```
def setCPF(self, cpf): self.CPF = cpf
```

```
def getCPF(self): return self.CPF
```

Na Programação Orientada a Objetos o conceito de herança é muito utilizado. Basicamente, dizemos que a herança ocorre quando uma classe (filha) herda características e métodos de uma outra classe (pai), mas não impede de que a classe filha possua seus próprios métodos e atributos. Na **Figura 2** podemos ver, em nível de diagrama, como esta relação ocorre:

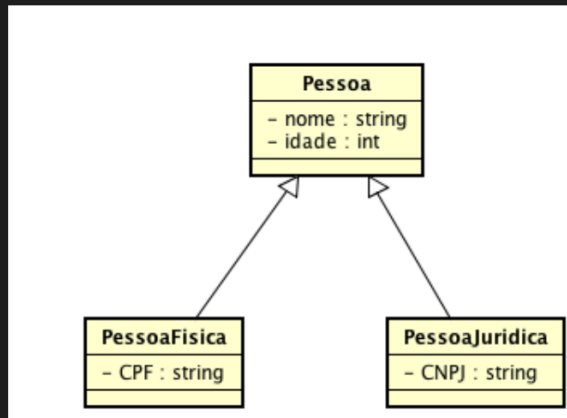


Figura 2. Diagrama de classes com PessoaFisica e PessoaJuridica herdando da classe Pessoa

Abaixo podemos ver como esta relação ocorre em Python:

```
1  from pessoa import Pessoa
2
3  class PessoaFisica(Pessoa):
4      def __init__(self, CPF, nome, idade):
5          super().__init__(nome, idade)
6          self.CPF = CPF
7
8      def getCPF(self):
9          return self.CPF
10
11     def setCPF(self, CPF):
12         self.CPF = CPF
```

Linha 1: Como vamos herdar da classe `Pessoa` e ela foi definida em outro arquivo (`Pessoa.py`), precisamos importá-la. Para isso usamos a instrução `import` e indicamos o nome do arquivo, sem a extensão `.py`, seguido do nome da classe que queremos importar;

Linha 3: Para definir que uma classe herdar de uma outra classe, precisamos indicar o nome da classe pai entre parênteses após o nome da classe filha;

Linha 4: Criamos o construtor da classe filha e definimos quais atributos ela espera receber. Neste caso, o nome e idade estão definidos na classe pai, enquanto o cpf é próprio da classe filha;

Linha 5: Utilizando o método `super`, invocamos o construtor da classe pai. Com isso aproveitamos toda a lógica definida nesse método, que no caso faz a atribuição dos valores de nome e idade aos atributos da classe. Com isso garantimos que ao ser criada, a classe filha efetuará o mesmo processamento que a classe pai e mais alguns passos adicionais.

Dessa mesma forma podemos criar a classe `PessoaJuridica`:

```
1  from pessoa import Pessoa
2
3  class PessoaJuridica(Pessoa):
4      def __init__(self, CNPJ, nome, idade):
5          super().__init__(nome, idade)
6          self.CNPJ = CNPJ
7
8      def getCNPJ(self):
9          return self.CNPJ
10
11     def setCNPJ(self, CNPJ):
12         self.CNPJ = CNPJ
```

Observe que o nome das classes começa com letra maiúscula (inclusive quando há mais de uma palavra, como em PessoaFisica). Essa é uma convenção de escrita de código na linguagem, mas não uma obrigatoriedade da sintaxe

Sugestão de conteúdo